

CleanAVLVisualizer.java - Final Presentation Document

1. Next / Previous Button Functionality

Code Snippet:

```
// Inside AVLTree class
private List<Node> traversallList = new ArrayList<>();
private int currentIndex = -1;

void generateTraversallList() {
    traversallList.clear();
    inOrder(root);
    currentIndex = -1;
}

private void inOrder(Node node) {
    if (node == null) return;
    inOrder(node.left);
    traversallList.add(node);
    inOrder(node.right);
}

Node getNext() {
    if (traversallList.isEmpty()) return null;
    if (currentIndex < traversallList.size() - 1) currentIndex++;
    return traversallList.get(currentIndex);
}

Node getPrev() {
    if (traversallList.isEmpty()) return null;
    if (currentIndex > 0) currentIndex--;
    return traversallList.get(currentIndex);
}
```

Explanation: - `traversallList` stores all nodes in *in-order sequence*. - `generateTraversallList()` performs an *in-order traversal* of the AVL tree. - `getNext()` and `getPrev()` allow navigation between nodes sequentially. - This logic powers the **Next** and **Previous** buttons, allowing the user to visually traverse the tree structure step-by-step.

Human Explanation (for teachers):

"Ma'am/Sir, these methods control the navigation part of the tree. The in-order traversal ensures that we move from the smallest to the largest element. When I click Next or Prev, it highlights each node in that order without changing the structure — like a slideshow of tree elements."

2. Integration of Next / Prev Buttons in the Control Panel

Code Snippet:

```
JButton nextBtn = new JButton("Next");
JButton prevBtn = new JButton("Prev");

styleButton(nextBtn, new Color(255, 152, 0));
styleButton(prevBtn, new Color(255, 87, 34));

control.add(nextBtn);
control.add(prevBtn);

nextBtn.addActionListener(e -> {
    avl.generateTraversallList();
    Node next = avl.getNext();
    if (next != null) {
        next.popping = true;
        drawPanel.highlightNode(next);
    }
});

prevBtn.addActionListener(e -> {
    avl.generateTraversallList();
    Node prev = avl.getPrev();
    if (prev != null) {
        prev.popping = true;
        drawPanel.highlightNode(prev);
    }
});
```

Explanation: - Two **buttons** (**Next** and **Prev**) are added to the control panel. - Each button uses an **ActionListener** to respond to clicks. - On click, the AVL tree traversal list is generated, and the **next** or **previous** node is highlighted visually. - **styleButton()** gives consistent color and hover effects for professional UI.

Human Explanation (for teachers):

"Here I created the actual buttons that control traversal. Each button uses an ActionListener to call my traversal functions. When clicked, the button tells the AVL tree to fetch the next or previous node and highlight it — which helps visualize movement through the tree."

3. Rotation Log System

Code Snippet:

```
private List<String> rotationLog = new ArrayList<>();

private void recordRotation(String type, String reason) {
    rotationLog.add("Rotation: " + type +
        "\nPurpose: Rebalance the tree so heights differ by at most
1." +
        "\nWhy: " + reason + "\n");
}

String getLastRotationExplanation() {
    if (rotationLog.isEmpty()) return "No rotation required for the last
operation.";

    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < rotationLog.size(); i++) {
        sb.append("Step ").append(i + 1).append(": \n");
        sb.append(rotationLog.get(i));
        if (i != rotationLog.size() - 1) sb.append("\n");
    }
    return sb.toString();
}
```

Explanation: - Keeps track of **every rotation performed** in the AVL tree. - Each entry in the `rotationLog` describes the **type of rotation**, its **purpose**, and **why** it was needed. - `getLastRotationExplanation()` prepares a readable explanation displayed in a `JTextArea` below the visualization.

Human Explanation (for teachers):

"This part keeps a record of every rotation that happens when the tree balances itself. For example, if a node is inserted and the balance factor becomes uneven, it logs the type of rotation like LL or RR and explains the reason. This helps visualize how AVL trees maintain balance."

4. Balance Factor Implementation

Code Snippet:

```
int balance(Node n) {
    return n == null ? 0 : height(n.left) - height(n.right);
}

void updateHeight(Node n) {
    if (n != null)
        n.height = 1 + Math.max(height(n.left), height(n.right));
}
```

Explanation: - The **balance factor** of a node = `height(left subtree) - height(right subtree)`. - A valid AVL node must have a balance factor of **-1, 0, or +1**. - `updateHeight()` recalculates heights after every insertion or deletion. - If any node's balance factor goes beyond ± 1 , rotations are triggered to restore balance.

Human Explanation (for teachers):

"The balance factor is the key concept of an AVL tree. It shows how balanced the node is. If it's +2 or -2, the tree becomes unbalanced and needs rotation. My code automatically checks this after every insertion or deletion and updates the height accordingly. That's how self-balancing is achieved."

5. Java Swing Components and Layout Overview

Components Used: - **JFrame:** The main window. - **JPanel:** Used for drawing (tree visualization) and control panels. - **JButton:** For actions like Insert, Delete, Next, Prev. - **JTextField:** To input values for insertion and deletion. - **JTextArea (inside JScrollPane):** Displays rotation explanations. - **BorderLayout:** Positions major panels (NORTH, CENTER, SOUTH). - **ActionListener:** Detects and handles button click events. - **Graphics & Graphics2D:** Used in `paintComponent()` for drawing nodes and edges. - **SwingUtilities:** Ensures UI updates run on the Event Dispatch Thread.

Human Explanation (for teachers):

"Swing is used here for GUI building. I've used JFrame as the main window, and JPanel for layout organization. Buttons and text fields are part of the control section, while Graphics2D helps draw the actual AVL tree. Everything runs safely on the Swing event thread using SwingUtilities."

6. One-Page Oral Script (For Presentation)

Opening:

"Good morning, respected panel. I worked on the visualization and interaction part of our AVL Tree project. My main contribution includes adding navigation features, balance factor visualization, and a rotation log system."

Part 1 — Next/Prev Functionality:

"I implemented in-order traversal storage and two buttons — Next and Previous — that help navigate the tree visually. It helps users move through nodes step by step without changing the tree structure."

Part 2 — Balance Factor:

"The balance factor tells us whether the tree is stable. My code automatically updates it after each insertion or deletion. If the factor exceeds 1 or -1, a rotation is triggered."

Part 3 — Rotation Log:

"Each rotation is recorded and explained — the type, the reason, and its purpose. This gives clarity about how AVL trees balance themselves."

Part 4 — Swing Components:

"I used JFrame, JPanel, JTextArea, and JButton to design the interface. The BorderLayout organizes these sections clearly, and custom drawing is done with Graphics2D."

Closing:

"In short, my work connects the logic and visualization — it helps users not only see but understand how AVL Trees balance themselves dynamically."